

RaceDay System Planning and Database

Module: Programming 2B

Module Code: PROG6212

Assessment Type: POE (Part 1)

Student Name: Reesaido Alagiry

Student Number: ST10335260

Table of Contents

Section	Page Numbers
System Overview	3
Section A: Entity Relationship Diagram (ERD)	4 & 5
Section A (continued) – Full size ERD	6
Section B – API Endpoint Plan	7, 8, 9 & 10
Proof of Submission	11
References	12

System Overview

RaceDay is a race-event management system that allows Organisers to create and manage running events, and allows Participants to browse events, enrol in a race category, pay their entry fee, and view their results once a race has taken place.

The system supports two roles:

- Organiser - creates and manages events and race categories, views who has enrolled, and captures race results.
- Participant - browses events, enrolls in a category, tracks their own enrolments, and views their own results.

Section A - Entity Relationship Diagram (ERD)

The ERD reflects the full data model for the RaceDay system. It contains eight entities: Roles, Users, Events, Categories, EventCategories, Enrolments, Results, and WeatherInfo. Primary keys (PK), foreign keys (FK), and the cardinality of every relationship are labelled directly on the diagram (see the full-size version on the following landscape page).

Entity Summary

Entity	Purpose	Keys
Roles	Lookup table for the two roles used by the system: Organiser and Participant.	PK RoleID
Users	Stores every user account. RoleID says whether they are an Organiser or Participant.	PK UserID, FK RoleID -> Roles
Events	A race event created and owned by one Organiser.	PK EventID, FK OrganiserID -> Users
Categories	A reusable master list of race distances (e.g. 5km, 10km, Half Marathon) shared across events.	PK CategoryID
EventCategories	Links a Category to a specific Event, with that event's entry fee and participant cap.	PK EventCategoryID, FK EventID -> Events, FK CategoryID -> Categories
Enrolments	A Participant enrolling into one EventCategory.	PK EnrolmentID, FK EventID -> Events, FK ParticipantID -> Users, FK EventCategoryID -> EventCategories
Results	The finishing result captured for a single Enrolment (one-to-one).	PK ResultID, FK EnrolmentID -> Enrolments
WeatherInfo	Optional forecast information linked to an Event.	PK WeatherID, FK EventID -> Events

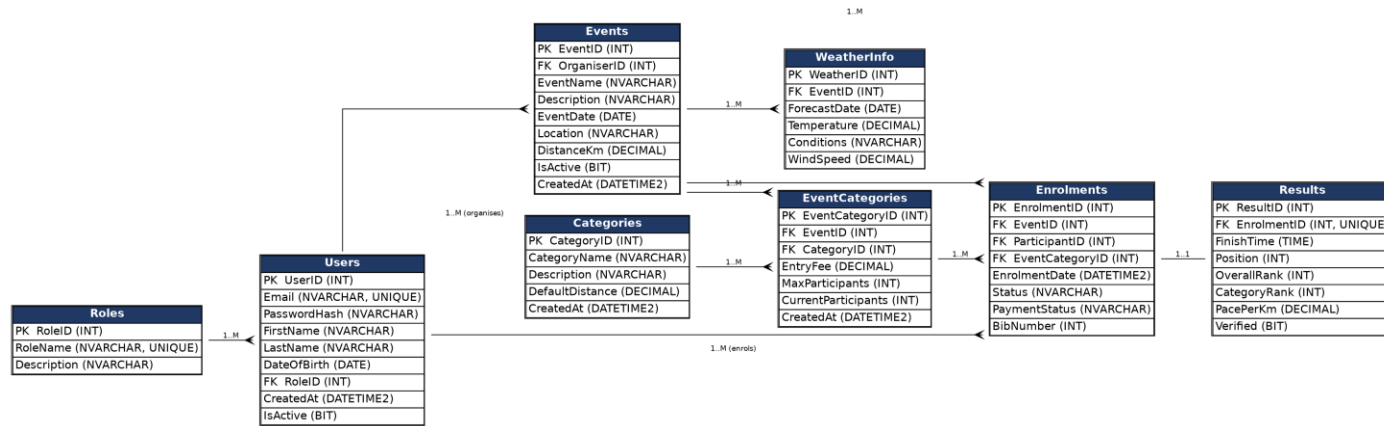
Relationship Notes

- Roles -> Users: one-to-many. One Role (Organiser or Participant) applies to many Users.
- Users -> Events: one-to-many. One Organiser can create many Events.
- Events -> EventCategories: one-to-many. One Event can offer many categories.

- Categories -> EventCategories: one-to-many. One master Category (e.g. 10km) can be offered at many events.
- Events -> Enrolments: one-to-many. One Event can have many Enrolments.
- Users -> Enrolments: one-to-many. One Participant can make many Enrolments (across different events/categories).
- EventCategories -> Enrolments: one-to-many. One EventCategory can have many Enrolments, up to its MaxParticipants.
- Enrolments -> Results: one-to-one. Each Enrolment has at most one Result, captured after the race.
- Events -> WeatherInfo: one-to-many. One Event can have one or more forecast updates.

Note: the ERD is also submitted separately as RaceDay_ERD.png inside the /docs folder, and the SQL script in Section C / RaceDay_Database.sql matches this ERD exactly.

Section A (continued) - Full-size ERD



Section B - API Endpoint Plan

The table below lists every API endpoint the RaceDay system will expose, covering Authentication, User Profile, Events, Categories, Event Enrolments, and Results, as required. Each endpoint's implementation in Part 2 must closely match this plan; any deviations will be explained in the README.

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
POST	/api/auth/register	Registers a new user as an Organiser or Participant.	None (public)	{ email, password, firstName, lastName, dateOfBirth, role }	201 Created - new user (no password returned); 400 - validation error; 409 - email already registered
POST	/api/auth/login	Authenticates a user and returns an access token.	None (public)	{ email, password }	200 OK - { token, user }; 401 - invalid credentials
GET	/api/users/me	Returns the profile of the currently logged-in user.	Any (logged in)	None	200 OK - user profile; 401 - not authenticated
PUT	/api/users/me	Updates the logged-in user's own profile details.	Any (logged in)	{ firstName, lastName, email }	200 OK - updated profile; 400 - validation error
GET	/api/events	Lists all upcoming events.	None (public)	None	200 OK - array of events
GET	/api/events/{id}	Returns full details for one event, including its categories.	None (public)	None	200 OK - event object; 404 - event not found
POST	/api/events	Creates a new event owned by the logged-in organiser.	Organiser	{ eventName, description,	201 Created - new event; 400 - validation error

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
				eventDate, location, distanceKm}	
PUT	/api/events/{id}	Updates an event the organiser owns.	Organiser (owner)	{ eventName, description, eventDate, location }	200 OK - updated event; 403 - not the owner; 404 - not found
DELETE	/api/events/{id}	Deletes an event the organiser owns.	Organiser (owner)	None	204 No Content; 403 - not the owner; 404 - not found
GET	/api/categories	Lists the master list of reusable race categories.	None (public)	None	200 OK - array of categories
GET	/api/events/{eventId}/categories	Lists the categories offered at a specific event, with fee and spots remaining.	None (public)	None	200 OK - array of event categories; 404 - event not found
POST	/api/events/{eventId}/categories	Links an existing category to an event with an entry fee and participant cap.	Organiser (owner)	{ categoryId, entryFee, maxParticipants }	201 Created - new event category; 404 - event/category not found
PUT	/api/event-categories/{id}	Updates the entry fee or participant cap for an event category.	Organiser (owner)	{ entryFee, maxParticipants }	200 OK - updated event category; 404 - not found

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
DELETE	/api/event-categories/{id}	Removes a category offering from an event.	Organiser (owner)	None	204 No Content; 404 - not found
POST	/api/event-categories/{id}/enrolments	Enrols the logged-in participant into an event category and assigns a bib number.	Participant	None	201 Created - enrolment with bib number; 404 - not found; 409 - already enrolled or full
GET	/api/users/me/enrolments	Lists all events/categories the logged-in participant has enrolled in.	Participant	None	200 OK - array of enrolments
GET	/api/event-categories/{id}/enrolments	Lists all participants enrolled in an event category (owner only).	Organiser (owner)	None	200 OK - array of enrolments; 403 - not the owner
DELETE	/api/enrolments/{id}	Cancels the logged-in participant's own enrolment.	Participant (owner)	None	204 No Content; 403 - not the owner; 404 - not found
POST	/api/results	Captures a finish time and position for a completed enrolment.	Organiser	{enrolmentId, finishTime, position}	201 Created - new result; 404 - enrolment not found
GET	/api/event-categories/{id}/results	Returns the public leaderboard for an event category, ordered by finishing position.	None (public)	None	200 OK - ordered array of results

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
GET	/api/users/me/results	Returns the logged-in participant's own race results.	Participant	None	200 OK - array of results
PUT	/api/results/{id}	Corrects a previously captured result.	Organiser	{ finishTime, position }	200 OK - updated result; 404 - not found

This plan will be submitted separately as a PDF/Markdown file (RaceDay_API_Plan) inside the /docs folder, as required by the brief.

Proof of Submission

The completed repository and video walkthrough for Part 1 can be found at the links below.

GitHub Repository:

https://github.com/ST10335260/ST10335260_PROG6212_POE_Part1.git

Video Walkthrough (YouTube, unlisted): <https://youtu.be/RxXAerlaFFw>

The repository's /docs folder contains the ERD (RaceDay_ERD.png), the API endpoint plan (RaceDay_API_Endpoint_Plan.pdf), the SQL database script (RaceDay_Database.sql), and a screenshot confirming a successful CI/CD run (RaceDay_ci_build_success.png). The README.md describes the system, the two user roles, and links to each of these documents.

References

Chen, P.P.S., 1976. The Entity-Relationship Model - Toward a Unified View of Data. ACM Transactions on Database Systems, 1(1), pp.9-36.

Codd, E.F., 1970. A Relational Model of Data for Large Shared Data Banks. Communications of the ACM, 13(6), pp.377-387.

GitHub, 2026. GitHub Actions Documentation. [online] Available at: <https://docs.github.com/actions> [Accessed 22 Sep. 2026].

Microsoft, 2026. Transact-SQL (T-SQL) Reference. Microsoft Learn. [online] Available at: <https://learn.microsoft.com/sql/t-sql/language-reference> [Accessed 22 Sep. 2026].

Microsoft, 2026. SQL Server Management Studio (SSMS) Documentation. Microsoft Learn. [online] Available at: <https://learn.microsoft.com/sql/ssms> [Accessed 22 Sep. 2026].

Richardson, L. and Ruby, S., 2007. RESTful Web Services. Sebastopol, CA: O'Reilly Media.